STEM PAPER

PROBLEM / PURPOSE STATEMENT

As technology continues to evolve exponentially, it's critical for security to stay on par with these advancements. A foundational factor of security involves random numbers, which are used for encryption, tokens, authentication, etc. Pseudorandom Number Generators (PRNGs) are commonly used to quickly produce accessible random numbers, but this method brings forth many security risks such as easily predictable seeds with low entropy. On the other hand, True Random Number Generators (TRNGs) provide true randomness but are less accessible to the general public and harder to acquire. This project aims to develop an accessible random number generator while mimicking true randomness by utilizing environmental sensors to enhance security.

BACKGROUND INFORMATION

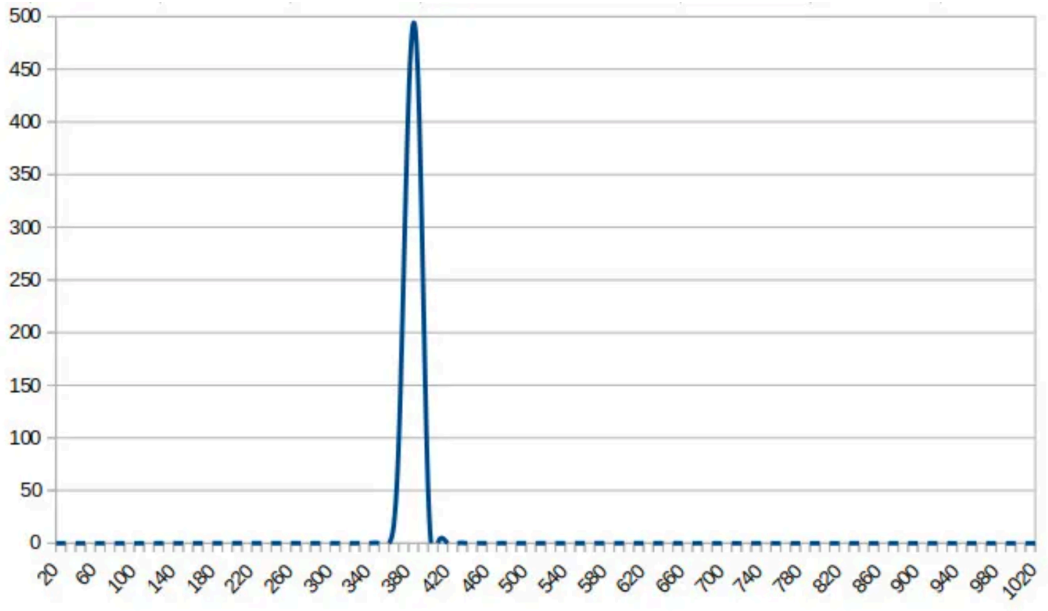
We are currently in an era known for its exponential growth of technology. While this brings many benefits, it also poses more security risks. Many that we have already encountered, and much more to come in the future. It's very important that security stays on the same level, if not go beyond, in order to keep not only our technology safe, but also us as users.

When people think of security, random numbers would be the unlikely thought of many. The importance of random numbers in the field of security is often overlooked. Many may not realize but random numbers are a big factor when it comes to encryption, tokens, passwords, etc (Sun, 2024). They ensure that data is protected and not easily accessed by outsiders. Though, it's important to note that while random numbers can protect, they can also be the reason why stronger protection is needed. Just having a set of random numbers won't automatically ensure quality of security. To put in perspective, according to Constantin (2021), researchers found defects in RSA keys and discovered that one in every 172 was at risk to security attacks and the cause was faulty generation of random numbers. It's prominent here that how those numbers are generated can either make or break the efficiency of how they perform.

Some may ask, what's the difference in each process of generating random numbers? One of the most common ways a random set of numbers can be generated is by using a pseudo generator. This method does not provide the highest security but it gets the job done quickly, which makes it accessible to more people. The main difference between pseudo random number generators (PRNG) and other more reliable generators is that PRNGs are

deterministic and predictable because PRNGs often rely on algorithms. The set of numbers can easily be recreated if the starting seed or input is to be found and then feeded into the same algorithm (Sidhpurwala, 2019). This makes the system more vulnerable to hackers which increases the risk of data leaks.

Pseudo random number generators are actually the system that Arduino, this project's control group, utilizes to create its own random numbers. Similarly to the findings above, Arduino's random() function does not ensure quality randomness in its numbers. A test was done in order to see the distribution of feeded seeds into the random() function algorithm by using inputs, as seeds, from data via an analogRead() pin. Results showed that there was a lack of widely ranged distributed seeds (List, 2018). Given that the PRNGs are deterministic and can recreate numbers as long as the same input is feeded into the algorithm, the chances that a set of random numbers gets duplicated aren't near zero and may unknowingly increase.

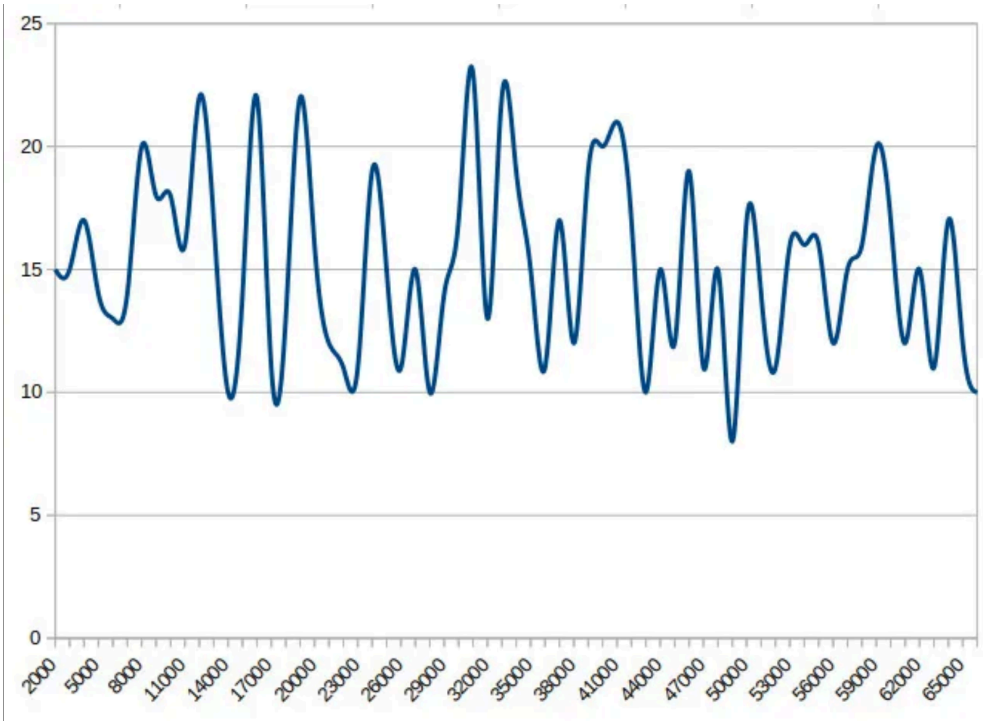


"The not-very-random result of a thousand analogRead() calls."
(List, 2018)

While PRNGs are the risky method, there is always an alternative and more safer method, which are True Random Number Generators (TRNG). These generators are self explanatory: they create numbers with true randomness. But what is true randomness? According to Sun (2024), "True randomness refers to unpredictable data that cannot be determined by an algorithm." Unlike PRNGs, TRNGs aren't deterministic, which significantly

decreases the chances of number replication. Looking back at Arduino's random() function, which uses data from analog pins, TRNGs use something different. TRNGs use physical sources like noise from the atmosphere or circuits to generate numbers which have been found to be unpredictable (Sun, 2024). One significant difference in both processes is that PRNGs use software inputs while TRNGs utilize hardware inputs.

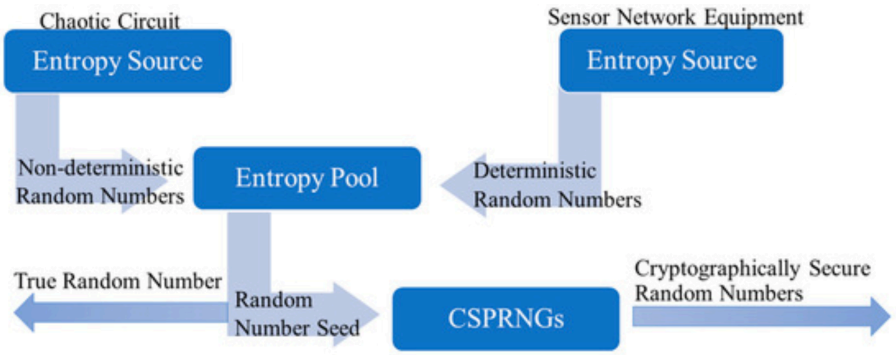
Referring back to the Arduino random() function experiment a few paragraphs prior, another experiment was conducted soon after. However, this time, a hardware source was utilized instead of Arduino's built in software generator. In the experiment, random numbers were created using seeds retrieved from jitters from a hardware microcontroller clock (List, 2018). The result came out significantly more random than before, when a software generator was used. The difference between software RNGs (PRNG) and hardware RNGs (TRNG) became prominent.



"The significantly more random result of a thousand Arduino Entropy Library calls" (List, 2018)

Behind all of these generators, there's one main idea all behind it: entropy. Entropy measures the randomness (disorder) of a system. The higher the entropy, the less predictable and certain the result is (Sidhpurwala, 2019). The reason the TRNG worked better was because

it produced a higher level of entropy. This level is typically higher when using the environment and atmosphere. Softwares usually have trouble producing quality entropy which reflects on the inputted seeds getting feeded into the algorithm.



“Random number generation model based on entropy pool”
(Zhang & Wu, 2023)

Environmental sensors are one of the most reliable sources of entropy, which leads to why I chose to utilize them in this project. Zhang & Wu (2023) states that “Environmental sensors (temperature, humidity, pressure, light, etc.) possess the capability to perceive the ambient environmental conditions. Due to various factors (weather fluctuations & human interference), these environmental conditions often exhibit inherent randomness.” Trying to recreate environmental conditions perfectly is closer to impossible than trying to simply guess which data input was inputted into an algorithm. To scientifically go in depth on why environmental sensors are more reliable, these sensors possess a physical phenomenon called Brownian Motion. Brownian motion is the constant movement of never still particles which causes collisions with other particles. The motions of the particles are difficult to observe due to its unpredictable paths and motion. This can be exemplified using temperature: As temperature increases, particles gain more energy to move and collide (*Brownian Motion,* 2024). Temperature sensors are one of the sensors that this project will utilize. Similarly, other sensors like light and sound provide similar results in producing entropy due to its state of unpredictable particle motion.

All things considered, utilizing environmental sensors can closely mimic True Random Number Generators by producing a nearer level of entropy that Pseudo Random Number Generators fail to produce efficiently. One con of TRNGs is that they may not be as accessible as PRNGs, which is why most people turn to the easier but less effective process when creating random numbers. While Arduino, typically used as a PRNG through the random() function, may

not be known to be reliable, certain Arduino sensors can be used to feed high-level entropy seeds into the same algorithm. The expected results are fairly similar to what would be outputted by a TRNG, all while being easily accessible and acquirable.

PROPOSED SOLUTION

The solution proposed through this project aims to shy away from using pseudo-generated seeds to generate random numbers. This project's generating system will instead utilize a commonly accessible microcontroller called an Arduino Uno along with environmental sensors to generate feedable seeds for the Arduino generator. For added security, the generated numbers will only be attainable through a terminal interface which can be accessed using passcodes. All will be stored in a 3D printed compartment containing the sensors. Since the numbers are dependent on the surroundings, the portable compartment allows the user to be in full control of the environment the numbers are being generated in.

HYPOTHESIS

Seeds derived from high-entropy sources such as the environment, will yield higher entropy and less deterministic random numbers compared to when a pseudo-generated seed is used instead.

MATERIALS USED

- Arduino Uno
- Breadboard / Mini-Breadboard
- Jumper Wires
- LM393 Sound Detection Sensor
- DHT11 Temperature Sensor Module
- Photoresistor
- Tactile Button
- 3D Print Filament

SOFTWARE USED

- Arduino IDE
- IntelliJ: Java, Spring Boot
- Supabase: PostgreSQL
- PyCharm: Python
- macOS Terminal
- GitHub: Version Control

PROTOTYPE DESIGN & DESCRIPTION

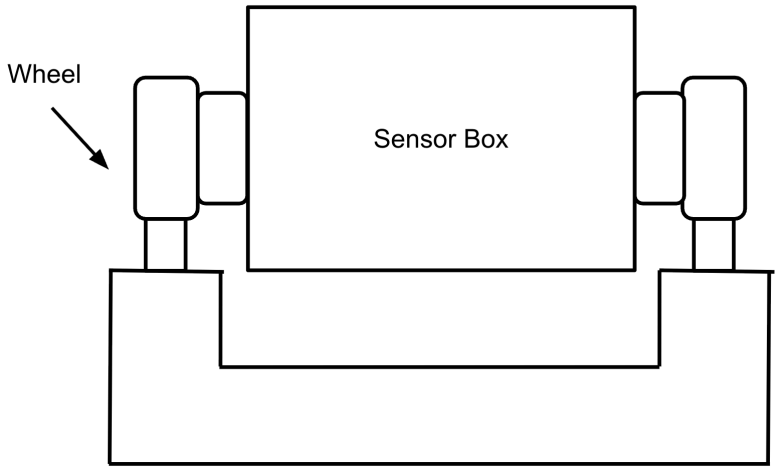


Figure 1.0

Prototype #1: Visual Aid

Prototype #1

The first prototype shown above had a main goal of capturing data in alternating ways in order to produce entropy. This was hoped to be accomplished by the spinning sensor box positioned in the middle of two wheel motors controlled by two buttons. The initial design stemmed from the idea that physically repositioning the sensors would allow the data to be captured more randomly. While the initial idea seemed practical, soon after, technical and theoretical flaws emerged, which resulted in a redesign. First, the rotary motion of the wheels and sensors caused problems when it came to connecting the Arduino Uno stationed within the Sensor Box to power. An alternative option was to utilize a battery pack, but the risk of quickly burning out the batteries emerged. Another option that looked good on paper but failed in application was an external wall plug. Unfortunately, the designs of the circuit board and power chord that was available for use really limited the rotary of the device. A common problem faced was the chord twisting and having to be untwisted every so often. This defect, if kept in the design, could've been the cause of many more future problems and would have defeated the purpose of simplicity and efficiency that this project is aiming to achieve.

Additionally, the initial idea of repositioning sensors was lacking. Simple repositioning would have mostly resulted in only slight alterations in the data. This is due to the fact that the sensors would still be capturing data from the same environment. Unless the surrounding

environment experiences notable change, the data won't be altered enough in between each trial. Thus, a redesign was conducted to achieve a more efficient performance from the device.

Prototype #2

In this final prototype, simplicity and efficiency were achieved in a better manner than prior. As seen in Figure 2.0 below, a more simple approach was taken to stay true to the project's goal. In the prior prototype, the user was able to control the device in hopes to alter the data. In this final and more portable version, the user can easily alter the surroundings by physically moving the device around and to certain settings or by altering the surroundings near the sensors. This approach will ideally produce more randomness in the data captured by the sensors. Additionally, a lot of possible technical hassles were eliminated by simplifying the design while staying efficient to meet the goal all with a single push of a button.

To go into depth about the mechanics of the device, the three sensors will be situated in its designated slots through the front wall of the box. The larger opening near the bottom right is to allow access to the button, which initiates the process of capturing and sending data. Figure 2.1 represents the Arduino Uno circuit board diagram.

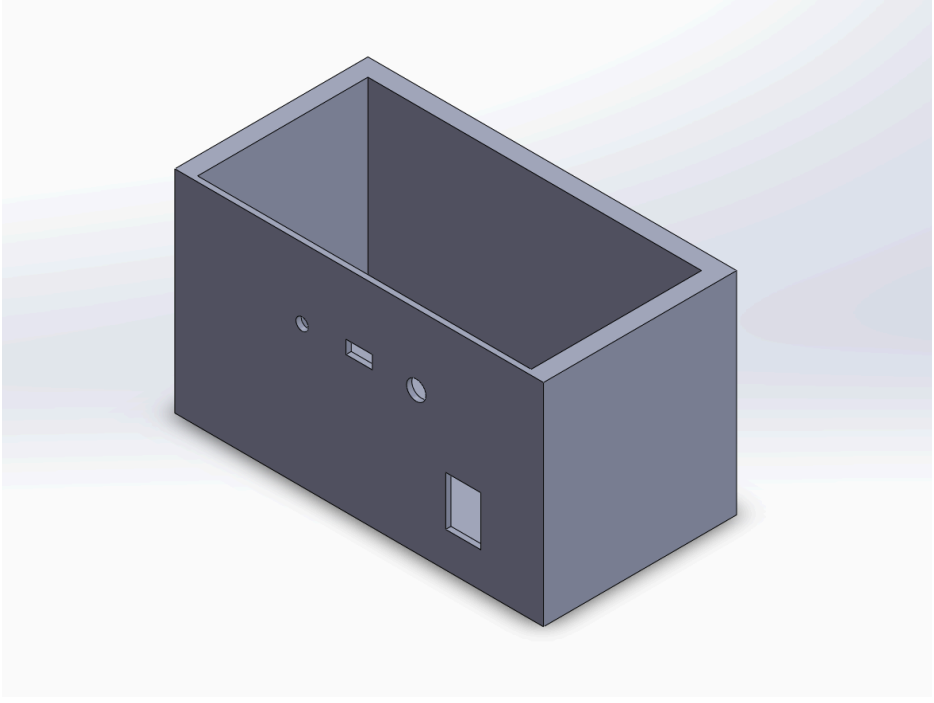


Figure 2.0

Prototype #2: 3D Model

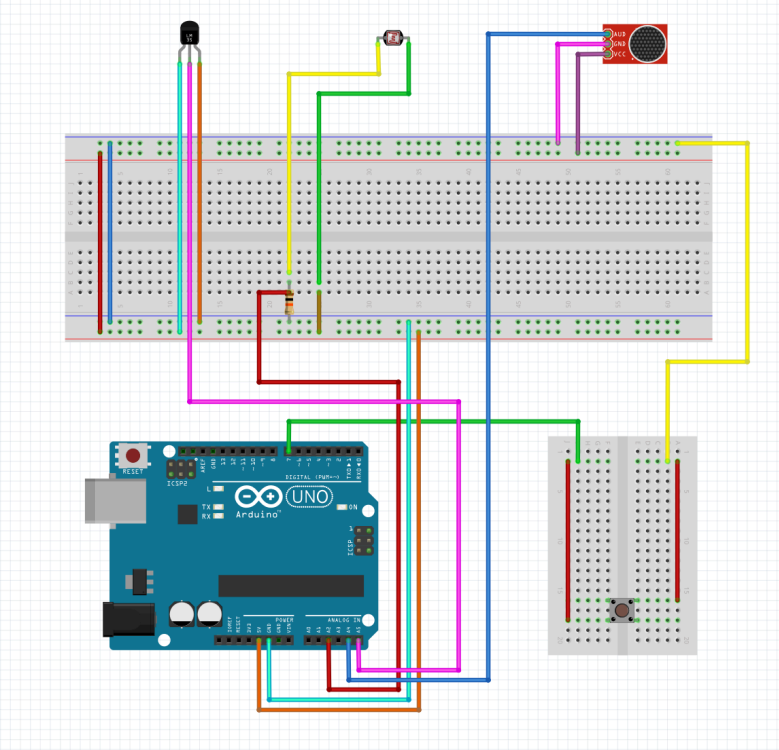


Figure 2.1

Prototype #2: Arduino Diagram

Following the press of the button, the software within this device handles the fetching, transferring, and storing of data. A Spring Boot Architectural layout was utilized to organize the data flow throughout. Once connected to a laptop, the device utilizes a terminal interface where the user must input a passkey to proceed with generating and accessing numbers. If the inputted key is successful, the terminal initiates the Spring Boot (Java) program and allows the remaining functionalities to play out. The user is then able to press the button on the Arduino breadboard which prompts the Arduino IDE to collect sensor readings which it then will send to the Spring Boot program via serial port. A Java library called jSerialComm will then process the data into the program. After being processed, the data will then be encrypted with AES (Advanced Encryption Standard) for added security. With this feature, even if an outsider were to gain access to the database, the actual contents of the database will stay encrypted and secured unless the passkey were to be also inputted. The persistence layer acts as the final bride, sending the data through a repository, then to the PostgreSQL database using a JDBC driver. See Figure 2.2 below for a visualization.

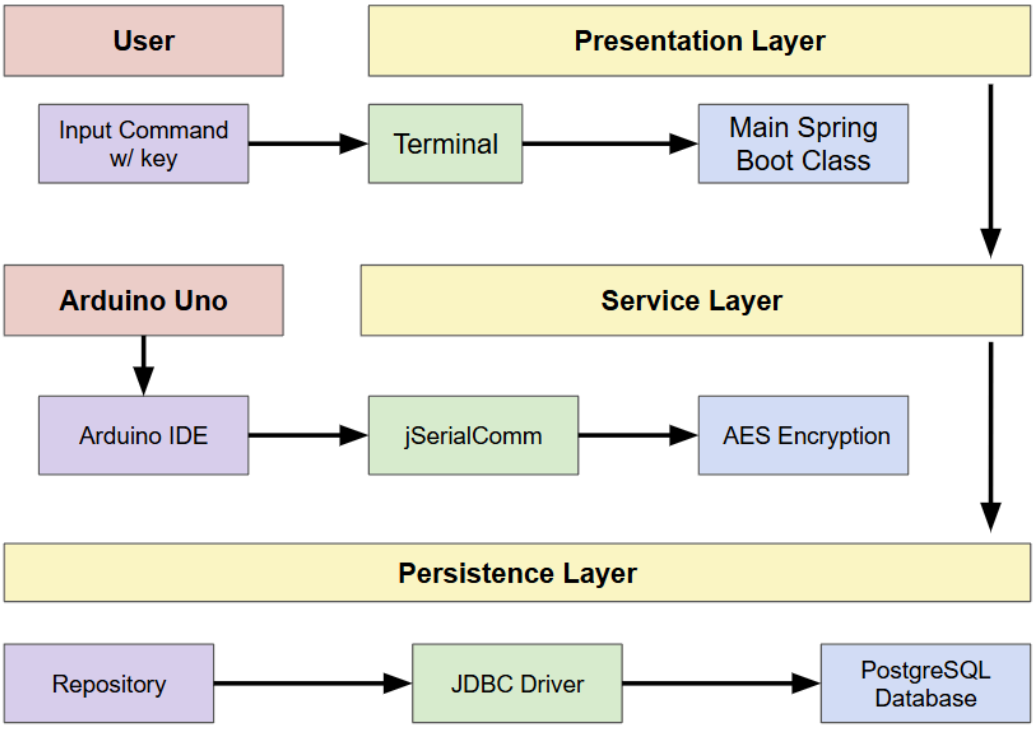


Figure 2.2

Prototype #2: Software Architecture

PROCEDURES

- Appropriately connect components and sensors to breadboard and Arduino Uno
- Orient components in a 3D printed box to correspond with designated openings.
- Connect Arduino Uno to the Arduino IDE software
- Write code in Arduino IDE to take readings from sensors, generate numbers, etc.

Software

- Create a Spring Boot/Java project through the Spring Initializer website with desired settings and dependencies
- Open project in Intellij (and connect to designated GitHub repository for version control)
- Set up packages/folders for specific Spring Boot layers
- Create PostgreSQL database through Supabase
- Connect PostgreSQL database to Spring Boot project in Intellij
- Create and code corresponding entity and repository classes for each table in database

- Create and code service classes to accommodate fetching data from Arduino IDE, conducting encryption, and storing in PostgreSQL database
- Conduct tests on macOS terminal to ensure smooth runs of program

VARIABLES

Independent Variable: Seeds used to generate random numbers

Dependant Variable: The random numbers generated

Control Group: Pseudo-Generated Numbers by the Arduino

Constant Group: Arduino's Random Generator Function [random()]

DATA ANALYSIS PROCEDURES

- Create a separate Arduino IDE program with similar functionalities as prior but with added features: generating a random number using Arduino's random() and pseudo-generated seeds
- Create a separate Spring Boot/Java project with similar functionalities as prior but with added features: fetching and storing the pseudo-generated number by Arduino. (Data isn't encrypted in the data analysis program version. This version utilizes a different table in the database compared to the final version)
- Create a Python project in Pycharm IDE and create separate scripts for each analysis method
- Code each script to pull/access data from database and generate charts to act as data visual aids
- Conduct each analyzation method through code via Python libraries: pandas, scikit, matplotlib

TESTING PROCEDURES

1. Connect Arduino Uno to laptop
2. Initiate Spring Boot (Java) program in Intellij
3. For each trial, alter sensor surroundings (adjust light, temperature, and/or sound)
4. Press button on Arduino breadboard to capture data
5. Check IntelliJ terminal to confirm data was fetched then stored in PostgreSQL database
6. Periodically check PostgreSQL database (via terminal or Supabase)
7. Repeat until desired threshold is reached

ANALYSIS METHODS

Frequency Test

Purpose: Checking for uniformity

Each number within the range should have around the same chance of appearing across the dataset. Determining how frequent each number appears across a hundred trials will conclude which system is more uniform in generating numbers. Ultimately, the more uniform, the better.

Value Over Time Test

Purpose: Checking for consistency

The trendline generated represents how the data acts over the timeframe it was tested on. The trendline will help determine whether the system stayed consistent or developed varying patterns of alterations along the way.

Monobit Test

Purpose: Checking for balance

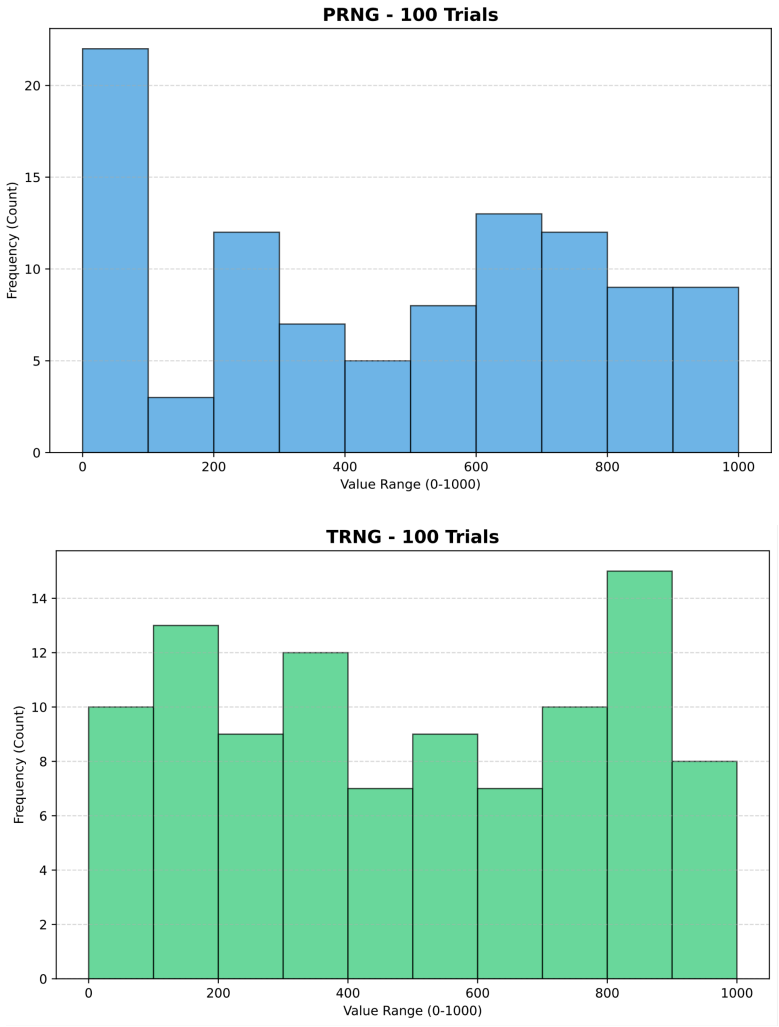
Similar to the frequency test, this test also checks for balance but along the lines or parity. Like a coin toss, there should be a 50-50 chance for each side to land upright. A successful generating system should come closer to a 50-50 ratio of odd and even numbers.

Chi-Squared Test

Purpose: Checking for relations

A successful system should act closer to the theoretical ideal system, meaning correlations and relations within the data are avoided, especially mathematical ones. Independency is the key to a better system. A higher P-value indicates a better consistency with a null system (theoretical ideal). A lower statistic score indicates being closer to a perfectly uniform system.

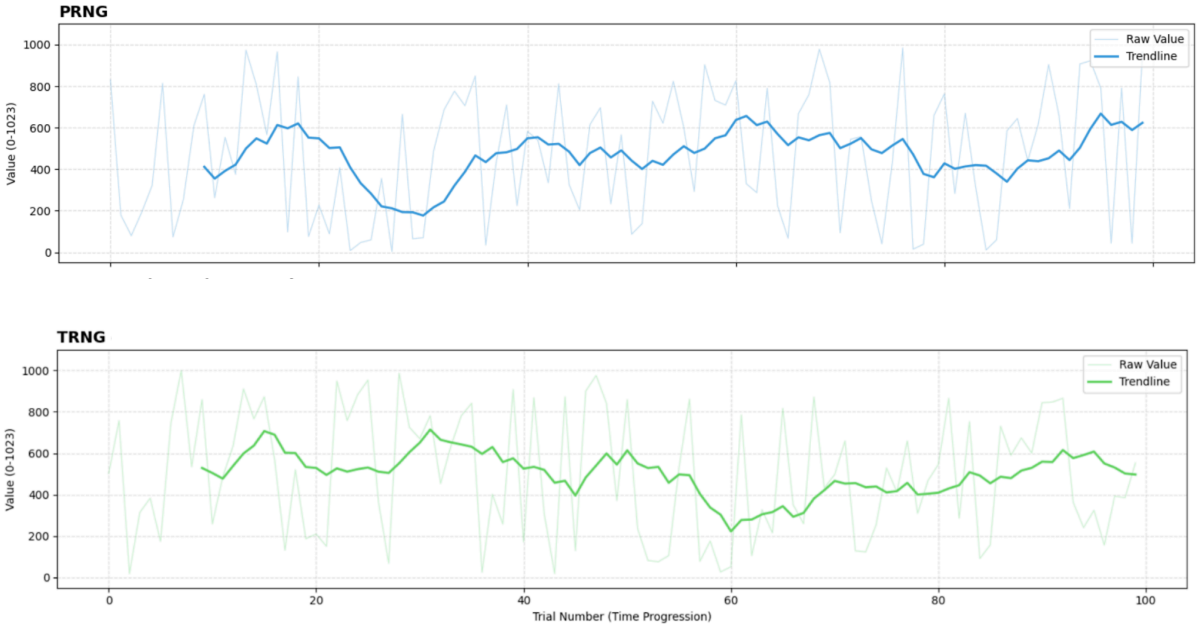
Frequency Test



Data Analysis: Frequency

Just by looking at the charts, the PRNG (number generated by the Arduino) clearly shows bias in certain ranges, with major spikes showing in the 0-100 range then significantly dropping in the 101-200 range. In comparison, the TRNG (number generated by this project's system) , showed a fairly more balanced output. There were minimal spikes and drops but the TRNG stayed shy of major changes unlike the PRNG. Straightforwardly, the TRNG produced numbers that were more fairly distributed than the PRNG. The risk that comes from the PRNG not being fairly distributed is that if the range that is favored by the system is determined, this will increase chances of pinpointing the number generated.

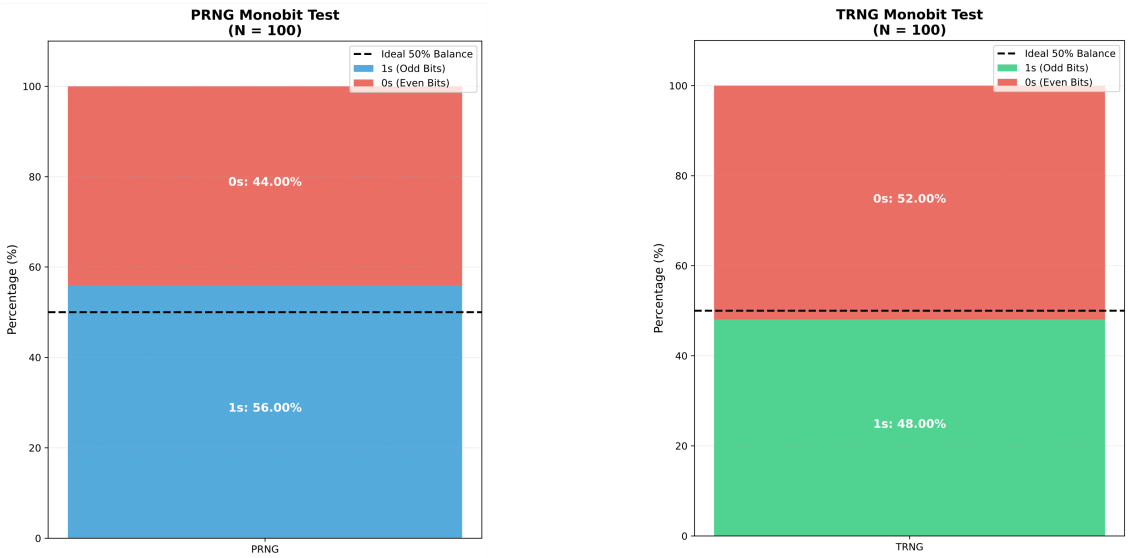
Value Over Time Test



Data Analysis: Value Over Time

It may be harder to distinguish which method performed better due to both trendline's similar paths. Both the PRNG and TRNG stayed near the halfway line throughout the majority of the timeframe. However, looking closely at trials 20-40, we see that the PRNG performed in a way that will tip the scales in the TRNG's favor. Within that range, the PRNG took a dip and did not recover for approximately after 10-15 trials. We can conclude from this dip that in that timeframe, the PRNG developed correlations. In other words, we assume that the generated data were influenced by the prior data, explaining why the trendline hovered around that specific level. These are relationships that occur more likely when a mathematical algorithm behind the system is present. Pattern recognition risks stem from relationship-based systems, which poses many security risks. Now evaluating the TRNG trendline, minimal dips do occur. However, the TRNG was able to recover faster than the PRNG and without resulting in any temporary influential levels in value. While the PRNG dip that was discussed prior only occurred once, take into account that it occurred once in a 100 trial test. If the range were to increase, the frequency of the dip may also increase. A security risk that could result from the trendline dip is being able to narrow down the possibilities of generated numbers and increasing the chance of correct guesses. When hovering at the 50% level, each number had an equal probability, but if the trendline was closer to one side, the scales clearly favor a range of certain numbers.

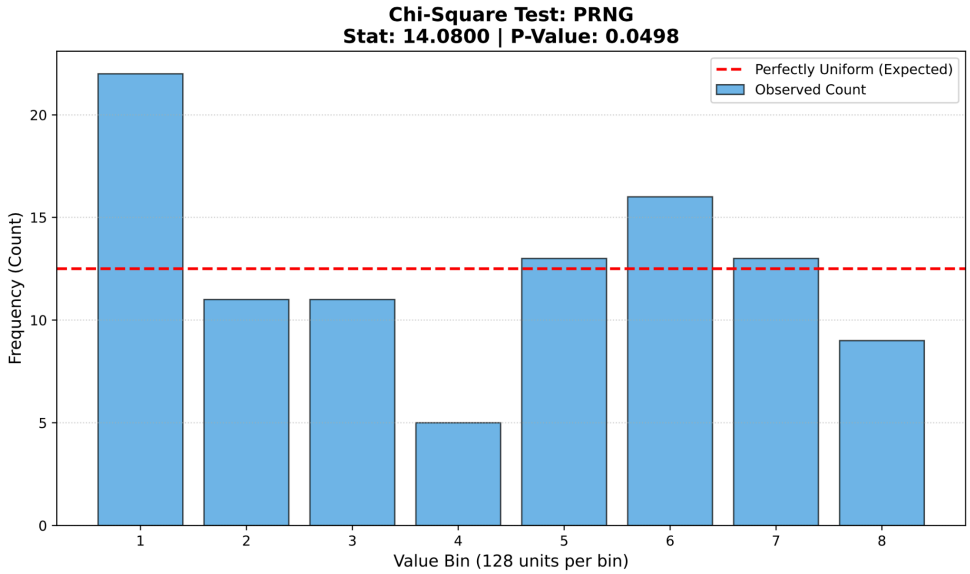
Monobit Test

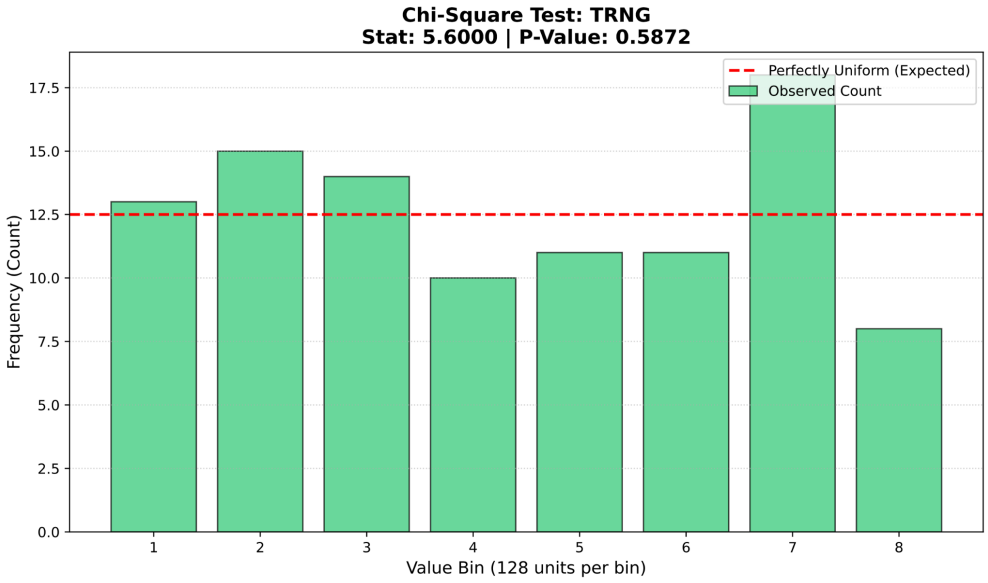


Data Analysis: Monobit Test

This test straightforwardly concludes that the TRNG performed better in generating numbers that had a ratio of odd-even numbers closer to 50-50. With only a 2% difference from the ideal level, the TRNG outperforms the PRNG which resulted in a 6% difference from the ideal level. Thus, providing more evidence that the TRNG is distributing numbers in a more fair and nondeterministic way.

Chi-Squared Test





Data Analysis: Chi-Squared Test

By looking at the numerical scores of each method, the TRNG has again performed better than the PRNG. The statistical score of the TRNG was 5.6, while the PRNG scored 14.08. This statistical score numerically represents the distance or difference between each method's data and a theoretical ideal data set, meaning that a lower score is better. The TRNG scored 86% better than the PRNG, meaning that the TRNG is performing similarly to an ideal case while the PRNG is performing less ideally. Additionally, this test also provided a P-value for both cases. The TRNG has a P-value of 0.58 while the PRNG has a P-value of 0.04. In this case, the higher P-value serves greater significance. Since the TRNG resulted in a higher P-value, thus, the numbers that the TRNG produced were more independent of one another and showed near zero correlation. PRNG's lower P-value shows that the number distribution was least likely to be occurring by luck and had influences tied to it. All in all, the TRNG performed greatly better than the PRNG.

CONCLUSION

To recall, this project's hypothesis claimed that random numbers derived from the environment are produced in a less deterministic way compared to pseudo-generated random numbers. Thus, the data and results concluded a successful hypothesis for this project. Whether it was by a small or large margin, the True Random Number Generator made with Arduino environmental sensors outperformed Arduino's built in pseudo random number generator as shown through the several tests.

Future Improvements

While the project's final prototype was enough to efficiently collect data, much more improvements can be made. First and foremost, since part of this project's goal was to achieve efficiency with simplicity, the user interface can be made to accommodate more people. The final prototype utilized a terminal interface but a better approach could've been a graphical user interface. Several obstacles prevented this. First, the code required leaned towards frontend development, while my skills concentrated on backend development. Time was also an issue as there were more implementations that could've been added if time allowed me to learn and practice certain coding skills needed. As for the project's physical component, more time spent working on it could've made a big difference and improvement. The overall design was basic but got the job done. More security features were also going to be implemented but technical and material issues regarding the circuit board prevented further implementations. Both the backend and physical part of the project brushed on basic methods and systems that can be improved to go more in depth. Ultimately, the final prototype represented a demo of a future version of the application and system.

REFERENCES

Brownian Motion. (2024, September 23). LabXchange. https://www.labxchange.org/library/items/lb:LabXchange:5e617e64:html:1#:~:text=What%20is%20Brownian%20Motion?,path%20is%20tracked%20in%20blue.&text=The%20steps%20the%20particle%20takes,away%20from%20its%20starting%20location.&text=Other%20names%20for%20Brownian%20motion,the%20system%2C%20and%20motion%20increases.

Constantin, L. (2021, August 13). *IoT devices have serious security deficiencies due to bad random number generation*. CSO Online; CSO. https://www.csoonline.com/article/571183/iot-devices-have-serious-security-deficiencies-due-to-bad-random-number-generation.html

List, J. (2018, January 8). *Entropy and the arduino: When clock jitter is useful*. Hackaday. https://hackaday.com/2018/01/08/entropy-and-the-arduino/

Sidhpurwala, H. (2019, June 5). Understanding random number generators, and their limitations, in Linux. *Red Hat Blog*. https://www.redhat.com/en/blog/understanding-random-number-generators-and-their-limitations-linux

Sun, R. (2024, September 12). *Randomness and Pseudorandomness in Encryption and Cybersecurity: A Focus on Mobile Development*. Medium. https://medium.com/@ssr-404/randomness-and-pseudorandomness-in-encryption-and-cybersecurity-a-focus-on-mobile-development-1fbf7266f9af

Zhang, J., & Wu, M. (2023). Random Number Generation Based on Heterogeneous Entropy Sources Fusion in Multi-Sensor Networks. *Distributed Observation and Control of Multi-Sensor Networks*. https://doi.org/10.3390/s23208497